

ICT3113 Assignment 1 Team Workplan

Item	Agreed planning basis
Official deadline	Friday 9 October 2026, 23:59 Singapore time
Internal finish	Thursday 8 October 2026, 18:00; upload preferably by 9 October, 18:00
Freeze before testing	Wednesday 30 September, 18:00: golden set and prediction record committed
Team number / CSV range	1000–1999; team 1 uses row IDs 1×1000 to $n \times 1000 + 999$ inclusive

Purpose. Build and measure a synchronous, CPU-only ticket triage baseline, then recommend a model based on the team’s requirements and reproducible evidence. The baseline has no cache or queue. Assignment 2 is where performance tuning belongs.

Owners

Owner	Accountable result	Main handoff
Ernest (P1)	Golden test set and labelling process	Frozen 175-ticket set, protocol, agreement and resolutions to Izzul; Slide 6
YP (P2)	Docker service, storage and instrumentation	Working API and reconcilable request logs to Izzul and Lutfi; Slide 2
Mikhail (P3)	Workload, requirements, predictions and synthesis	Signed-off load scenarios to Lutfi; requirement and recommendation logic; Slides 3–4, 11–12
Izzul (P4)	Pinned Ollama models, CPU environment and accuracy	Model pins to Mikhail; accuracy and confusion matrices to team; Slides 5, 7, 10
Lutfi (P5)	Open-loop JMeter, stress test and performance evidence	Run files, results and bottleneck to Mikhail; Slides 8–9

Dated team milestones

Date	Exit condition	Accountability
26 Sep	Confirm group registration, row range, team machines, repository, and roles. Set a short daily check-in.	All five
27 Sep, 12:00	Labelling protocol and sampled row IDs ready; API/log contract and model shortlist agreed.	Ernest, YP, Izzul
28 Sep, evening	All 350 independent labels complete; baseline API and workload draft ready; open-loop JMeter plan drafted.	All five
29 Sep, evening	Disagreements resolved; agreement computed; service and requirements reviewed; synthetic/non-golden dry run only.	Ernest, YP, Mikhail, Lutfi
30 Sep, 18:00	GOLDEN SET + PREDICTIONS FROZEN AND COMMITTED together; exact candidate pins recorded; commit hash saved.	All five sign off
1 Oct	First measured runs begin after freeze; CPU-only and separate load generator verified; request IDs reconcile.	YP, Izzul, Lutfi
4 Oct, evening	Accuracy, three-run load matrix and one stress test complete; all .jtl and service logs retained.	Izzul, Lutfi
6 Oct, evening	Results checked against requirements; prediction errors explained; recommendation agreed.	All five
7 Oct, evening	Complete slide draft (12 maximum) and all supporting files reviewed.	Mikhail + slide owners
8 Oct, 18:00	Rebuild, evidence audit and submission package complete.	All five
9 Oct, by 18:00	Upload and confirm submission; official cutoff 23:59.	Submitter + witness

Individual task breakdown (this is an ai generated recommendation , please do your own research and **READ THE SPEC**)

Ernest — golden set and labelling

27 Sep, 12:00 Protocol and sample. Define seven categories and rules for overlap, unclear narratives and apparent non-matches. Draw 175 reproducible team-row tickets; record seed, row IDs and provisional source strata.

27–28 Sep Independent labelling.

29 Sep Adjudication and statistic. Calculate nominal Krippendorff alpha and raw agreement on original labels. Discuss every disagreement with its annotators, document resolution and protocol revisions. If exclusion is unavoidable, record the rule and select a predeclared replacement before model results.

30 Sep, 18:00 Freeze. Export final 150–200 row IDs and labels (target 175). Commit golden set, sheets, resolution log, protocol and agreement with Mikhail's prediction record; save commit hash.

2–5 Oct Accuracy review and slide. Cross-check Izzul's correct/total and confusion counts. Invalid or failed model outputs must remain visible. Prepare Slide 6 and a difficult-category account for Slide 11.

YP — service and instrumentation

27 Sep Interface contract. Agree with Izzul and Lutfi on the seven category strings, request/run IDs, model switch procedure, logging schema and persistent storage. Pin software dependencies.

28–29 Sep Baseline implementation. Build Docker Compose service and CPU-only Ollama connection. Implement POST /tickets (one narrative, synchronous classification, store and return category), GET /search and GET /stats. Start empty; never bulk-import CSV. Check persistence, search, counts and error paths.

30 Sep Observability and rebuild. Log every handled request, including route, timestamp, run/request IDs, model pin, status, prediction or error, end-to-end time and model-call time when measurable. Rebuild on the selected CPU machine; document reset steps and hardware/software.

1–4 Oct Test support. Match Lutfi's .jtl IDs to service logs, investigate failures and queueing symptoms, and keep service version/config consistent. If a bug fix changes comparability, repeat affected runs.

7 Oct Architecture slide. Prepare Slide 2 with triage service, Ollama, storage and separate load generator; document all three endpoints and baseline synchrony.

Acceptance: Another member can rebuild with Compose, run all endpoints, and trace each reported request to a service log. No public inference API or performance optimisation.

Mikhail — workload, requirements and recommendation

27–29 Sep Quantitative workload. Find and cite relevant public complaint-volume figures. Explicitly estimate daily tickets, peak/non-peak arrivals, agent searches and transaction mix. Measure ticket-length distribution only within the team's 1,000 rows; record formulas, assumptions, units and sensitivity.

29–30 Sep Requirements and prediction record. Set testable response-time percentile, sustained classification throughput, and overall/per-category golden-set accuracy requirements under named loads, including peak. Collect Izzul's per-model accuracy/single-request latency predictions, YP/Lutfi's bottleneck prediction, and Ernest's hardest categories. Make each claim falsifiable; commit by 30 Sep 18:00.

1–4 Oct Evidence monitoring. Ensure Lutfi's tested arrival rates follow the signed-off workload; track source figures, run IDs and analysis outputs for each slide number. Avoid altering frozen predictions in light of results.

5–7 Oct Decision and deck. Compare each candidate with every requirement; explain prediction misses, trade-offs, unmet thresholds and limitations of scaling student hardware to client deployment. Lead Slides 3–4, 11–12 and integrate the full deck.

8 Oct, 18:00 Final audit. Check units, denominators, load conditions, references, names/IDs and GitHub link. Validate every numeric claim against raw results or an explicit workload calculation.

Acceptance: Every workload figure is reproducible from cited inputs and assumptions; every requirement has a pass/fail test; the recommendation follows the measured results.

Izzul — models, CPU environment and accuracy

27–28 Sep Candidate set. Choose 3–5 feasible Ollama models across at least two parameter size classes. Record exact tag and digest, size, licence and reason for inclusion; give pins to Mikhail before predictions freeze.

29–30 Sep Fair evaluation setup. Fix one classification prompt, generation settings and category parser across candidates. Confirm CPU-only inference. Document service, Ollama and separate generator machines, network, software and scaling limitations. Smoke-test only on synthetic/non-golden tickets before freeze.

1–4 Oct Accuracy experiment. After freeze, POST every golden ticket to the service for every candidate. Keep row ID, ground truth, prediction, model pin, request ID and error. Reset/isolate storage for equivalent model conditions. Compute overall accuracy, each gold category's correct/total and confusion matrix.

5–6 Oct Interpretation and slides. Review counts with Ernest, describe significant category confusions and candidate trade-offs, and prepare Slides 5, 7 and 10. Give licensing citations to Mikhail.

Acceptance: Every accuracy figure can be reconstructed from the frozen golden row ID, model output, denominator and exact Ollama digest.

Lutfi — JMeter, stress test and performance evidence

28–30 Sep Playbook and dry run. Build a repeatable .jmx using Open Model Thread Group or Precise Throughput Timer. Specify team-row ticket data, mixed search traffic if tested, offered rates based on Mikhail’s model, warm-up/measurement periods, reset state, file names, stop rules and model switch. Validate on synthetic/non-golden traffic before freeze. Use a separate load-generator machine.

1–4 Oct Measured load matrix. For every candidate and planned arrival rate, perform three comparable open-loop runs. Cover non-peak, peak and above-peak/probing conditions where feasible. Preserve each raw .jtl and service log with run ID, model digest, settings and start/end time; record offered and achieved rates.

By 4 Oct Stress test. On at least one candidate, find a defensible limit such as unstable latency, throughput saturation or error onset. Predefine criterion, increment and stopping rule; capture CPU/memory and model timing where available.

5–6 Oct Analysis and slides. Calculate p50/p95/p99, achieved throughput and errors per run/configuration, plus means and spread over the three runs. Reconcile counts and IDs with YP’s logs. Report how failed requests enter each metric. Prepare Slides 8–9 and bottleneck diagnosis.

Acceptance: A tester could repeat each test without inventing parameters, and every reported value maps to retained .jtl and service log evidence.

Final delivery and review

Deliverable	Owner	Check
Slides 1–12; GroupNum.pptx	Mikhail integrates; each owner authors assigned slides	At most 12 slides, cover names/IDs and GitHub link, clear sources and traceable numbers
Golden-set supporting files	Ernest	Final row IDs/labels; protocol revisions; individual sheets; agreement and resolutions
Prediction record	Mikhail	Bottleneck, per-candidate accuracy/latency, hardest categories; pre-benchmark commit
Rebuildable repository	YP, Izzul, Lutfi	Compose and source; model pins; every .jtl; service logs; analysis and run instructions
Cross-review	Ernest ↔ Izzul; YP ↔ Lutfi; Mikhail ↔ team	Accuracy denominators; request reconciliation; requirement consistency; final decision

